

**CENTRALE
LYON**

ALGORITHMES COLLABORATIFS ET APPLICATIONS

Voyageur de commerce

Élèves :

Clément GUILHAUMON
Carl-Henri GEGOUT

Enseignants :

Alexandre SAIDI

31 mars 2025

Table des matières

Introduction	2
1 Résolution avec l'algorithme des fourmis	2
1.1 Principe biologique	2
1.2 Adaptation au TSP	2
1.3 Mise en oeuvre de l'algorithme	3
1.4 Optimisation des paramètres	4
2 Interface graphique	5
3 Conclusion	7

Introduction

Le problème du voyageur de commerce (TSP, pour Travelling Salesman Problem) est un problème classique d'optimisation combinatoire en mathématiques et en informatique. Il peut être formulé ainsi :

Un voyageur doit visiter un certain nombre de villes une seule fois chacune, puis revenir à son point de départ. Le but est de trouver le chemin le plus court (ou le moins coûteux) qui permet de visiter toutes les villes.

Il s'agit d'un problème de graphe, où les villes sont représentées par des nœuds et les trajets par des arêtes pondérées (les distances ou les coûts de déplacement). Il est NP-difficile, ce qui signifie qu'il n'existe pas d'algorithme efficace connu pour le résoudre de manière exacte dans un temps raisonnable pour un grand nombre de villes. Ce problème a de nombreuses applications pratiques, notamment en logistique, transport, planification et robotique.

1 Résolution avec l'algorithme des fourmis

1.1 Principe biologique

Les fourmis réelles utilisent des phéromones pour marquer les chemins menant à une source de nourriture. Lorsqu'une fourmi trouve un chemin plus court, elle le parcourt plus souvent et y dépose plus de phéromones, ce qui attire d'autres fourmis. Progressivement, le chemin optimal est renforcé et devient dominant.

1.2 Adaptation au TSP

Dans le contexte du TSP, on simule une colonie de fourmis cherchant à minimiser la distance totale parcourue. Les villes sont représentées par des nœuds. Les trajets possibles entre les villes sont des arêtes pondérées avec deux types d'informations :

1. Distance : la longueur du trajet entre deux villes.
2. Phéromone : une valeur numérique qui reflète l'attractivité du trajet pour les fourmis.

L'algorithme de recherche du plus court chemin dans le cadre du TSP prend alors la forme suivante. Au début de l'algorithme, on initialise la civilisation. Pour cela, on définit le graphe des villes. De plus, on définit un nombre de fourmis que l'on va envoyer sur le graphe. Ces fourmis commencent sur un nœud aléatoire du graphe. Puis, on itère jusqu'à atteindre stagnation (voir l'étape 4 ci-après) ou bien jusqu'à avoir atteint le nombre d'itérations souhaité. A chaque itération, on suit ces étapes :

1. Les fourmis parcourent une à une le graphe, c'est à dire qu'elles choisissent une ville vers laquelle aller jusqu'à ce qu'elles aient visité toutes les villes. Elles effectuent ces choix selon la distribution de probabilités suivante : $P(j) = \frac{\tau_{ij}^\alpha \cdot \eta_{ij}^\beta}{\sum_{k \in V} \tau_{ik}^\alpha \cdot \eta_{ik}^\beta}$ où $P(j)$ est la probabilité de choisir la ville j , η_{ij} est l'inverse de la distance entre les villes i

et j , τ_{ik} le niveau de phéromone sur l'arête reliant i et j , V l'ensemble des villes, α et β des hyperparamètres pondérant respectivement les importances des niveaux de phéromone et des distances dans les choix des fourmis.

2. Ensuite, en vue de la prochaine itération, on met à jour le niveau de phéromone de chaque arête reliant les villes. Tout d'abord, on modélise l'évaporation de la phéromone : pour chaque arête, reliant i à j , à l'itération k , on calcule : $\tau_{ijk+1} = \tau_{ijk} \times \rho$, où ρ est le taux de persistance de la phéromone (et $1 - \rho$ est le taux d'évaporation). Suite à cela, chaque fourmi dépose Q/L phéromone sur les arêtes qu'elles viennent de parcourir, avec Q un hyperparamètre modulant la quantité de phéromone déposée, et L la distance totale parcourue par la fourmi lors de cette itération.
3. Puis, on met à jour le plus court trajet trouvé en le comparant avec les trajets effectués pendant cette itération.
4. On vérifie d'autre part qu'il n'y a pas stagnation de l'algorithme, c'est à dire que les fourmis ne suivent pas toutes la même boucle (indépendamment de leur point de départ).
5. Enfin, pour chaque fourmi, on vide sa liste tabou (liste qui contient les villes que la fourmi a visité, dans l'ordre), on réinitialise la longueur de son parcours L (en la mettant à 0) et on lui choisit un nouveau point de départ aléatoirement.

On remarque que cet algorithme contient 5 hyperparamètres :

1. Le nombre de fourmis
2. α , positif, l'importance relative de la phéromone
3. β , positif, l'importance relative de la distance
4. ρ , la persistance de la phéromone, $0 \leq \rho < 1$
5. Q , paramètre lié à la quantité de phéromone déposée par les fourmis

On s'intéresse à optimiser l'algorithme en jouant sur ces hyperparamètres dans la section 1.4.

1.3 Mise en oeuvre de l'algorithme

Pour modéliser le graphe, on utilise la bibliothèque python networkx.

Pour modéliser la colonie de fourmis, nous avons programmé une classe python Civilisation (disponible dans le fichier "Civilisation.py"). Cette classe a plusieurs attributs :

1. le graphe, représentant le réseau des villes (qui sont toutes connectées entre elles)
2. un attribut pour chaque hyperparamètre
3. une liste contenant des instances de la classe Fourmi développée dans "Fourmi.py"
4. le plus court trajet obtenu ainsi que sa longueur

Cette classe est ainsi dérivée de la classe Fourmi, qui a plusieurs attributs pour gérer les hyperparamètres, les villes parcourues, la distance parcourue, et la civilisation à laquelle une fourmi appartient (la classe Fourmi dérive donc aussi de Civilisation).

Finalement, le tout est mis en oeuvre à travers une interface graphique développée dans le fichier "interface.py", ou encore dans le fichier "test.py" qui permet d'effectuer l'optimisation des hyperparamètres de l'algorithme comme on va le voir ci-après.

1.4 Optimisation des paramètres

Dans cette section, on s'intéresse à l'optimisation de l'algorithme en jouant sur les hyperparamètres détaillés dans la section 1.2. Voici le cadre que nous avons utilisé pour effectuer des simulations afin de rechercher les meilleurs hyperparamètres de l'algorithme :

- Le nombre m de fourmis est fixé égal au nombre N de villes. Ici, on choisit 30.
- Les positions des villes sont choisies aléatoirement (et fixées évidemment une fois choisies). L'échelle est arbitraire.
- Pour rechercher les meilleurs paramètres, on fait varier un paramètre tout en fixant tous les autres.
- On fixe le nombre d'itérations maximal à 60 (nombre d'itérations déterminés empiriquement, en regardant la convergence de l'algorithme pour différents hyperparamètres avec l'interface développée dans le fichier "interface.py").
- On fait la moyenne des résultats obtenus sur dix simulations.

Ce que l'on fait s'appelle un gridsearch. On cherche les paramètres parmi les ensembles suivants :

$$\alpha \in \{0, 0.5, 1, 2, 5\}, \quad \beta \in \{0.1, 2, 5\}, \quad \rho \in \{0.3, 0.5, 0.7, 0.9, 0.999\}, \quad Q \in \{1, 100, 10000\}$$

Le programme python mettant en oeuvre ce gridsearch est disponible dans le fichier test.py. Les résultats obtenus par les différentes simulations sont disponibles dans le fichier resultats_simulations.csv.

On remarque plusieurs choses : le paramètre Q a très peu d'influence sur le résultat (en effet, on ne fait que jouer sur l'échelle de la phéromone).

D'autre part, lorsque $\alpha = 0$ (i.e. on ne tient pas en compte des niveaux de phéromone) modifier ρ n'a pas d'influence sur le résultat (qui change tout de même légèrement dû au caractère non déterministe de l'algorithme), ce qui est évident et heureusement vérifié par notre programme.

La prise en compte des phéromones (ce qui revient à augmenter α) améliore progressivement les résultats. Cependant, pour $\alpha = 2$ et $\alpha = 5$, la stagnation arrive très vite. En effet, l'importance de la phéromone devient trop prédominante relativement à la distance, ce qui fait que les fourmis se mettent rapidement à suivre le même chemin. Cela dégrade la performance de l'algorithme par rapport à $\alpha = 1$.

En revanche, augmenter β jusqu'à 5 a augmenté les performances de notre algorithme dans tous les cas. Cela permet de prendre de plus en plus en compte la distance ; en effet, si l'on ne prend pas suffisamment en compte les distances, le choix des villes se rapproche d'un choix aléatoire selon une distribution uniforme, ce qui ne fournit pas de bons résultats.

Enfin, le taux ρ de persistance de la phéromone s'interprète de la même façon que le paramètre α . Un ρ trop faible réduit les performances car on ne prend pas assez en compte les phéromones ; notre modèle explore trop l'espace des solutions, il n'exploite pas assez ses expériences passées. Un ρ trop élevé réduit aussi les performances car on converge trop rapidement vers une solution ; on explore pas assez l'espace des solutions.

Finalement, notre gridsearch dans cet espace d'hyperparamètres est tombé d'accord avec le texte du BE, et a trouvé comme paramètres optimaux $\alpha = 1, \beta = 5, \rho = 0.5, Q = 100$. La figure 1 montre l'évolution de la moyenne des résultats autour de ces paramètres optimaux, correspondant à l'analyse faite ci-dessus (hormis pour Q qui a eu de l'influence ici).

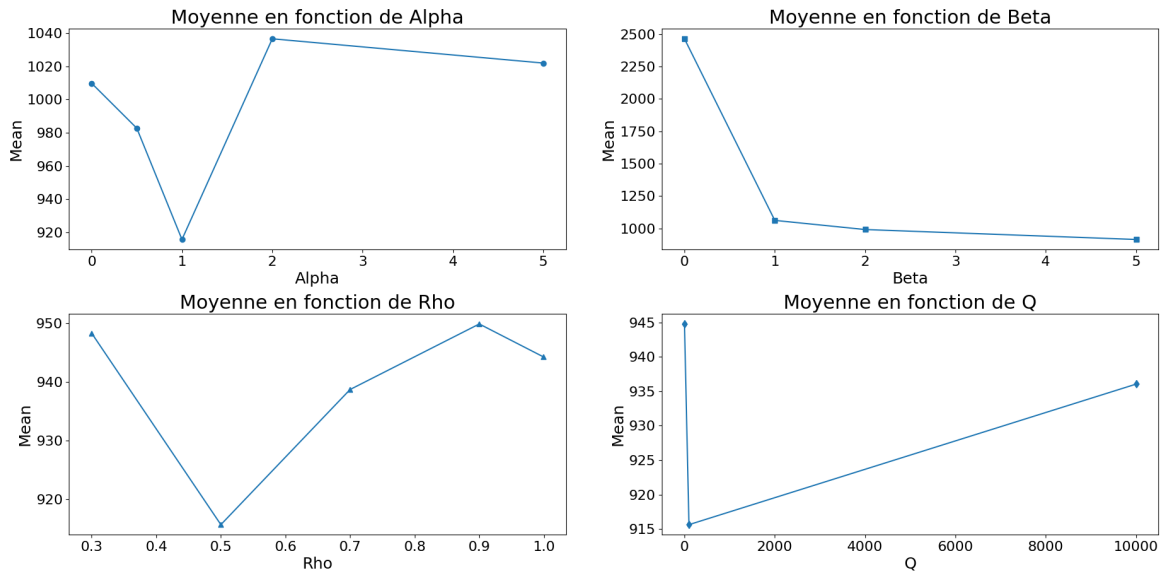


FIGURE 1 – Moyennes sur 10 simulations

2 Interface graphique

Une interface graphique a été réalisée avec la librairie python *Tkinter* (voir figure 2). Cette interface permet de placer les noeuds sur un plan et de régler différents paramètres (alpha, bêta ...) pour voir leur influence sur l'efficacité de l'algorithme.

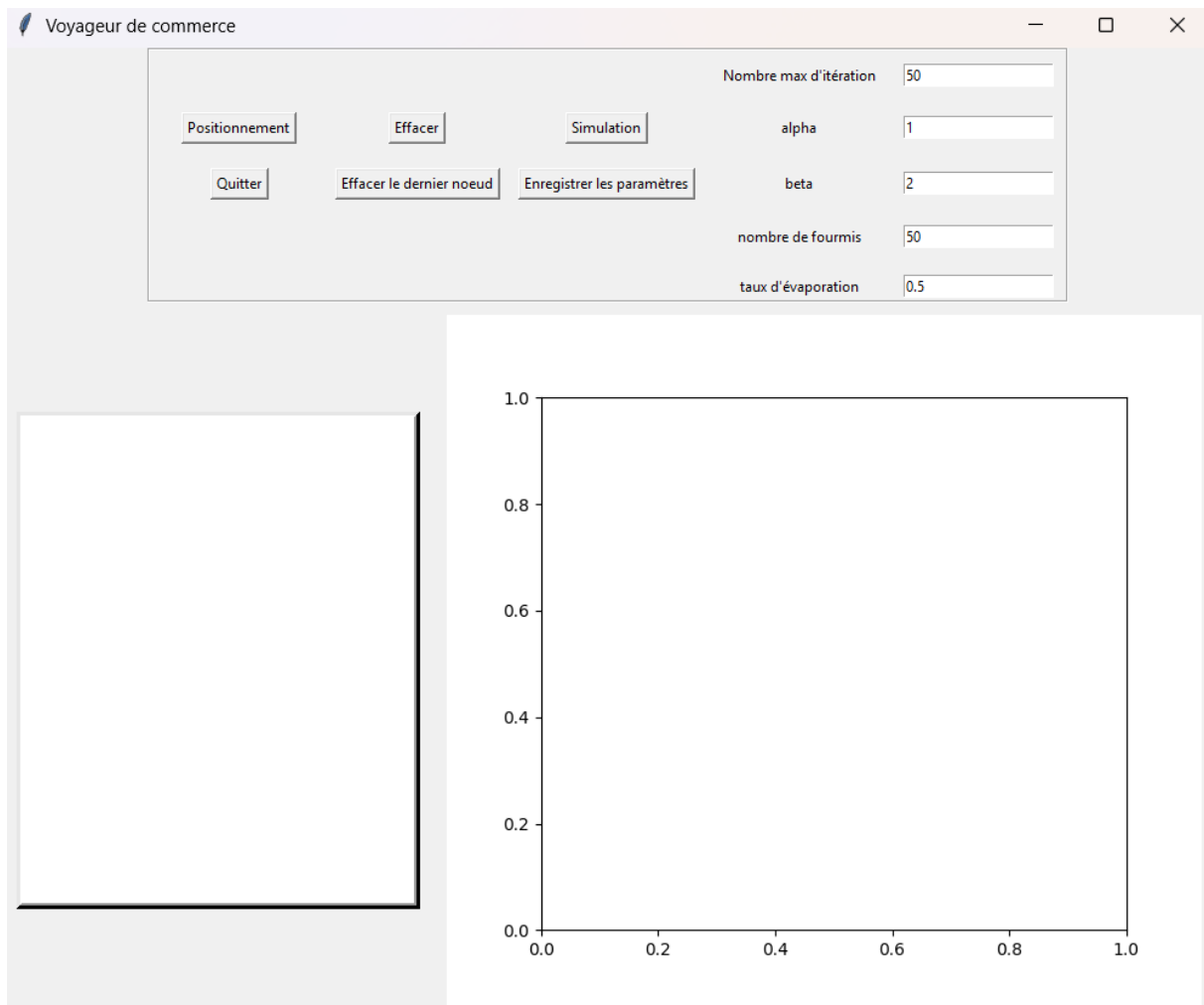


FIGURE 2 – Interface graphique

Une fois les noeuds placés et les paramètres saisis, la simulation peut être lancée avec le bouton *Simulation*. Cela change l’affichage, à la place de la zone pour placer les noeuds, le graphe avec le plus court chemin est dessiné. A droite, un autre graphe montre l’évolution du plus court chemin en fonction de l’itération (voir figure 3).

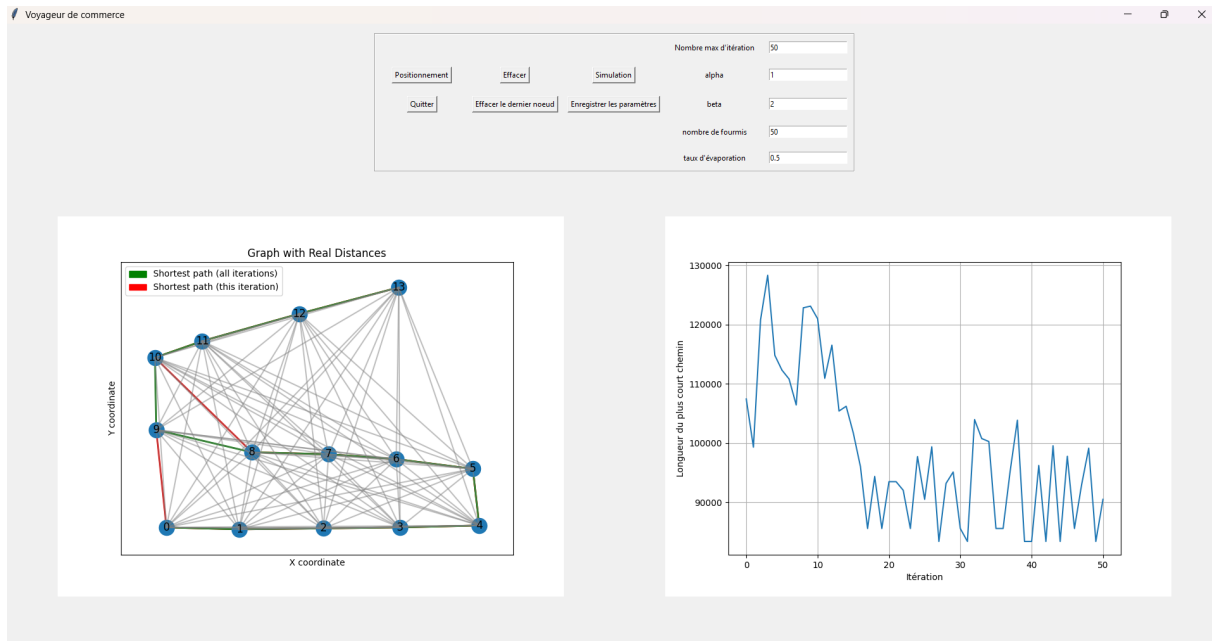


FIGURE 3 – Interface graphique en mode simulation

3 Conclusion

Pour conclure, nous avons pu proposer des solutions efficaces au problème du voyageur de commerce à l'aide d'un algorithme stochastique basé sur les colonies de fourmis, qui permet de trouver une solution satisfaisante sans être trop coûteux en calcul, en se basant sur la nature et notamment sur la stigmergie. Nous avons également pu optimiser cet algorithme en cherchant dans un sous-ensemble de ses hyperparamètres possibles, et nous avons pu développer une interface graphique permettant de visualiser le fonctionnement et la convergence de l'algorithme, tout en proposant à l'utilisateur de modifier dynamiquement le problème traité (en positionnant les villes en cliquant avec la souris), ainsi que les paramètres de l'algorithme.